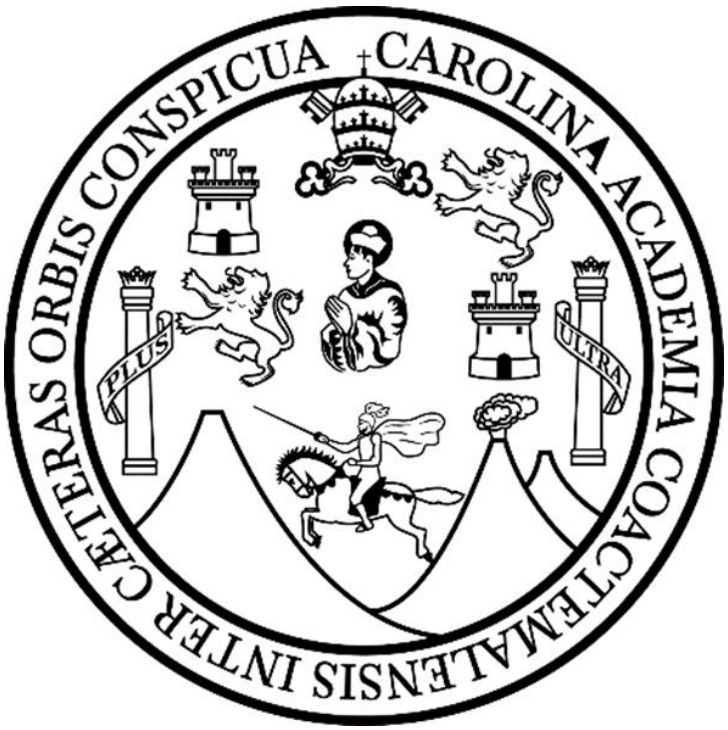
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Escuela de Ciencias y Sistemas



Manual Técnico – Informe 4 (Parte 1)

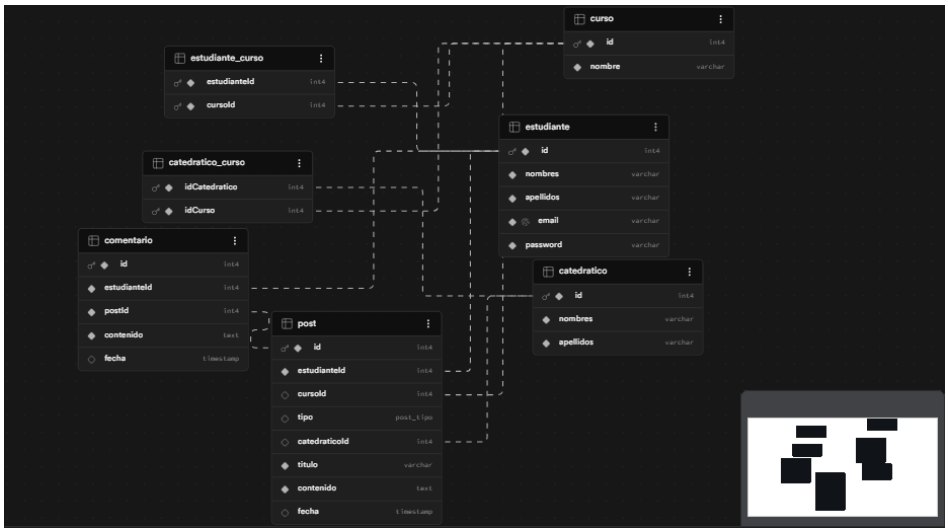
Ingeniería en Ciencias y Sistemas

Nombre: Jose Juan Laynez Zapeta

El proyecto, hasta donde va, consta de la base de datos en supabase y la parte del backend, con los endpoints para las consultas a la base de datos, hecho con node.js.

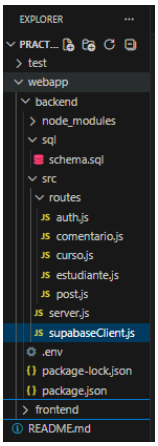
Base de datos:

Para la base de datos, se realizaron se realizaron 7 tablas. A continuación, se muestra el diagrama ER en supabase:



Las tablas catedratico_curso y estudiante_curso fueron hechas para trabajas una entidad de muchos a muchos entre curso y estudiantes y catedraticos. En caso se necesite tratar a un curso de manera individual, se puede hacer con la tabla curso.

De esta forma, ya con la base de datos en supabase, se trabajó en visual studio y la consola de comandos para crear la parte del backend, comenzando con la autenticación para que podamos conectar node.js con supabase.



En este caso, ya tenemos armado el backend, sin embargo, comenzamos con crear las carpetas vacías. Luego de instalar node.js, ejecutamos comandos como “npm install y-” “npm install express @supabase/supabase-js dotenv” y “npm install --save-dev nodemon”, configuramos los archivos generados y creamos el .env para crear la conexión.

```
webapp > backend > .env
PORT=5432
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOi
SUPABASE_URL=https://zndipwowmcnvcxdhoggl.supabase.co
DB_SSL=true
```

```
webapp > backend > package.json > ...
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "nodemon src/server.js",
    "test": "echo \"Error: error al conectar"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "@supabase/supabase-js": "^2.99.1",
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Posteriormente, se pudo conectar comprobar la conexión con el comando “npm run dev”.

```
C:\Users\lainz\Desktop\UNI\PracticasInicialesDump\PracticasIniciales_Informe4_fiusacWebapp\webapp>cd backend

C:\Users\lainz\Desktop\UNI\PracticasInicialesDump\PracticasIniciales_Informe4_fiusacWebapp\webapp\backend>npm run dev

> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
[dotenv@17.3.1] injecting env (4) from .env -- tip: ⚙ auth for agents: https://vestauth.com
Server running on port 5432
^C¿Desea terminar el trabajo por lotes (S/N)? S

C:\Users\lainz\Desktop\UNI\PracticasInicialesDump\PracticasIniciales_Informe4_fiusacWebapp\webapp\backend>npm run dev

> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
[dotenv@17.3.1] injecting env (4) from .env -- tip: ⚙ secrets for agents: https://dotenvx.com/as2
Server running on port 5432
```

Posteriormente, se trabajó con los endpoints en visual studio, lográndose realizar uno para registro y login, otro para creación y visualización de estudiantes, y otro para la creación de cursos.

```
webapp > backend > src > routes > auth.js > ...
const express = require('express');
const router = express.Router();
const supabase = require('../supabaseClient');

// POST reg
router.post('/registrar', async (req, res) => {
  const { id, nombres, apellidos, email, password } = req.body;
  const { data, error } = await supabase
    .from('estudiante')
    .insert([{ id, nombres, apellidos, email, password }]);
  if (error) return res.status(400).json({ error: error.message });
  res.json({ message: 'Estudiante registrado', data });
});

// POST login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const { data, error } = await supabase
    .from('estudiante')
    .select('*')
    .eq('email', email)
    .eq('password', password)
    .single();
  if (error || !data) return res.status(401).json({ error: 'Credenciales
  res.json({ message: 'Login exitoso', estudiante: data });
});

module.exports = router;
```

```
package.json M | package-lock.json | estudiante.js M | curso.js M | server.js M | auth.js M
webapp > backend > src > routes > curso.js > ...
const express = require('express');
const router = express.Router();
const supabase = require('../supabaseClient');

// GET curso
router.get('/', async (req, res) => {
  const { data, error } = await supabase.from('curso').select('*');
  if (error) return res.status(400).json({ error: error.message });
  res.json(data);
});

// POST curso

router.post('/registrar', async (req, res) => {
  const { id, nombre} = req.body;
  const { data, error } = await supabase
    .from('curso')
    .insert([{ id, nombre}]);
  if (error) return res.status(400).json({ error: error.message });
  res.json({ message: 'Curso registrado', data });
});
```

```
package.json M | package-lock.json | estudiante.js M | curso.js M | server.js M
webapp > backend > src > routes > estudiante.js > ...
const express = require('express');
const router = express.Router();
const supabase = require('../supabaseClient');

// GET
router.get('/', async (req, res) => {
  const { data, error } = await supabase.from('estudiante').select('*');
  if (error) return res.status(400).json({ error: error.message });
  res.json(data);
});

// GET from id
router.get('/:id', async (req, res) => {
  const { data, error } = await supabase
    .from('estudiante')
    .select('*')
    .eq('id', req.params.id)
    .single();
  if (error) return res.status(404).json({ error: 'Estudiante no encontrado' });
  res.json(data);
});
```

Estos se pudieron comprobar en postman:

http://localhost:5432/api/curso/registrar

POST http://localhost:5432/api/curso/registrar

Docs Params Authorization Headers 8 Body Scripts Sett

none form-data x-www-form-urlencoded raw bina

```
{
  "id": 771,
  "nombre": "Juan"
}
```

Body Cookies Headers 8 Test Results

{} JSON Preview Visualize

```
{
    "message": "Curso registrado",
    "data": null
}
```